TECHNICAL DOSSIER | V1.0.0 ARCHITECTURE SPEC | ACTIVE ENTERPRISE STACK

Digital Archaeologist

Intelligent Legacy Codebase Analysis, AST Intelligence & Software Maintenance Platform

DOMAIN	FRONTEND	BACKEND & DB	ANALYSIS ENGINE
Software Engineering / Maintenance	Next.js 16 • React 19 • Tailwind 4	Node.js / Express • SQLite • Prisma	Python 3.13 • Git AST Forensics

01 Executive Overview & Project Philosophy

"Understand existing software before changing it." In enterprise software engineering, between 60% and 80% of total lifecycle costs are expended on software maintenance, legacy onboarding, refactoring, and bug investigations. When senior engineers inherit multi-year repositories, they face steep cognitive hurdles: missing documentation, undocumented circular dependencies, hidden blast radiuses, volatile files (churn hotspots), and stale dead code.

Unlike generic AI tools that hallucinate high-level summaries without grounding, **Digital Archaeologist** provides a deterministic, AST-driven forensic platform. It reconstructs architectural topologies, measures cyclomatic complexity and Git churn velocity, detects dependency cycles via directed graph algorithms, and calculates transitive blast radiuses before a single line of code is edited.

Deterministic AST Engine

Multi-language parser extracting classes, methods, imports, and branch complexity without running unreliable test runners.

Transitive Blast Radius

Calculates upstream and downstream dependency ripple effects, SQL write impacts, and third-party gateway vulnerabilities.

Grounded AI Explanations

Contextual AI assistant that accompanies every answer with verified file paths, line numbers, snippets, and Git commit hashes.

02 High-Level System Architecture & Data Pipeline

Digital Archaeologist operates as a cohesive, 3-tier hybrid architecture backed by a persistent relational database:

Pipeline Orchestration Flow:
Target Repository Ingestion → Git History Scanner → Python AST Tokenizer → Directed Dependency Adjacency Graph → DFS Cycle Detector → Churn vs Complexity Matrix → Prisma / SQLite Storage → RESTful API → Real-time Next.js Developer Workspaces

Layer	Technologies	Responsibilities
Tier 1: Presentation	Next.js 16.3, React 19, Monaco Editor, React Flow (@xyflow/react), Recharts, Tailwind CSS v4	High-density developer dashboard, interactive graph canvas, VS-Code inspired 3-pane codebase explorer, scatter matrix, citation inspector.
Tier 2: API & Orchestration	Node.js, Express 4.19, TypeScript 5.5, Prisma ORM 6.19, tsx	RESTful gateway, subprocess runner for the Python engine, impact analysis calculations, AST-grounded local AI service, reporting generator.
Tier 3: Analysis Engine	Python 3.13, Regex AST Tokenizer, Git CLI Subprocess, Graph Adjacency	Multi-language source parsing, cyclomatic complexity counter, DFS cycle detection, Git commit & file volatility (churn) mining.
Storage Tier	SQLite, Prisma Client with 15 Relational Models	Normalized storage of analysis runs, file metadata, function & class definitions, dependencies, technical debt findings, and commit logs.

03 Deep Dive: Python AST & Forensic Engine (`analyzer.py`)

The core analysis engine located at `analysis-engine/analyzer.py` is a lightweight, zero-dependency Python script designed for maximum execution speed across heterogeneous codebases (TypeScript, JavaScript, Python, Java, Go, Rust).

1. Cyclomatic Complexity Calculation

Computes the number of linearly independent paths through program source code using McCabe's formulation based on decision points:

```
# Decision patterns extracted by regex:
patterns = [
    r'\bif\b', r'\belse\s+if\b', r'\belif\b', r'\bfor\b', r'\bwhile\b',
    r'\bcase\b', r'\bcatch\b', r'\bexcept\b', r'\b\?\s*[^:]+:', r'&&', r'\|\|'
]
complexity = 1 + sum(len(re.findall(p, content)) for p in patterns)
```

2. Directed Adjacency Graph & DFS Cycle Detection

Constructs both forward adjacency (`adjacency[u] → v`, meaning `u` depends on `v`) and reverse adjacency (`rev_adjacency[v] → u`, meaning `u` is a dependent of `v`). Circular dependencies are isolated via Depth-First Search with a three-color recursion stack:

- **0 (Unvisited):** Node has not been touched.
- **1 (Visiting):** Node is on the active traversal path. If an adjacent node has state `1`, a circular dependency cycle is detected.
- **2 (Visited):** Node and all reachable subtrees are completely analyzed.

3. Git Forensics & Churn Dynamics

Extracts repository commit history and file volatility using direct Git CLI bindings: `git log --pretty=format:%h|%an|%ad|%s` and `git log --name-only`. Files with high historical modification frequency are flagged as high-churn candidates.

4. Mathematical Composite Health Score

The overall Engineering Health Index (0–100) is synthesized using a multi-factor weighted equation:

```
Health Score = round(
    (ArchitectureScore * 0.25) +
    (MaintainabilityScore * 0.25) +
    (TestCoverageScore * 0.20) +
    (DependencyHealthScore * 0.20) +
    (DocumentationScore * 0.10)
)
Where:
- ArchitectureScore = max(20, 100 - (circular_count * 15) - (high_coupling_modules * 5))
- MaintainabilityScore = max(20, round(100 - (avg_cyclomatic_complexity * 3)))
- TestCoverageScore = min(100, round((test_files / total_files) * 300))
- DocumentationScore = min(100, round(20 + (doc_files * 15)))
- DependencyHealthScore = max(20, 100 - (technical_debt_items * 4))
```

04 Backend Architecture & RESTful API Endpoints

The Express TypeScript server (`backend/src/server.ts`) serves port `5000`, handling lifecycle actions, file browsing, impact analysis, and grounded AI queries:

Method	Endpoint	Controller & Purpose
`POST`	`/api/analyze`	Starts repository analysis pipeline (Git clone / local scan / Python engine invocation).
`GET`	`/api/analysis/status`	Polls active pipeline stage (Cloning → AST → Graph → Churn → Completed).
`GET`	`/api/repos/:id/overview`	Returns high-level metric lines, health index breakdown, and immediate attention alerts.
`GET`	`/api/repos/:id/architecture`	Generates React Flow node/edge graph with circular dependency highlights and module roles.
`GET`	`/api/repos/:id/dependencies`	Returns fan-in (dependents), fan-out (depends-on), and circular loops table.
`GET`	`/api/repos/:id/files`	Full directory tree hierarchy with LOC, language, complexity, and symbols.
`GET`	`/api/repos/:id/file-content`	Returns raw file contents for Monaco Editor with line metrics and symbol maps.
`GET`	`/api/repos/:id/git-history`	Commit logs, author distributions, and most volatile files (churn ranking).
`GET`	`/api/repos/:id/tech-debt`	Categorized debt inventory (Complexity, Dead Code, Security/MD5, Circular Loops).
`GET`	`/api/repos/:id/risk-analysis`	Scatter plot data points mapping Change Frequency (X) vs Cyclomatic Complexity (Y).
`POST`	`/api/repos/:id/impact-analysis`	Calculates transitive blast radius, database write operations, and call hierarchy tree.
`POST`	`/api/repos/:id/ai/query`	Deterministic AI responses with file paths, line ranges, code snippets, and commit hashes.
`POST`	`/api/repos/:id/docs/generate`	Generates structured Markdown documentation and architectural audit reports.

05 Static Blast Radius & Transitive Impact Analysis

Implemented in `backend/src/services/impact.service.ts`, this subsystem solves the classic developer dilemma: *"If I modify function X, what breaks across the rest of the application?"*

Downstream & Upstream Traversal

Traverses the reverse adjacency graph to discover direct and transitive callers. If `PaymentService.processPayment()` is modified, the service discovers that `CheckoutController`, `TransactionService`, and `NotificationService` are impacted.

Side-Effect & Infrastructure Risk

Scans call paths for database write operations (e.g., `DatabaseConnector.executeWrite()`) and external third-party gateway interactions (Stripe, PayPal, banking APIs) to establish risk ratings (`CRITICAL`, `HIGH`, `MEDIUM`, `LOW`).

Impact Output Schema

```
interface BlastRadiusResult {
  targetSymbol: string;
  targetFile: string;
  affectedModules: string[];
  affectedFunctions: string[];
  databaseOperations: number;
  externalApis: number;
  riskRating: "CRITICAL" | "HIGH" | "MEDIUM" | "LOW";
  confidence: "High (Static AST)" | "Requires Verification";
  callHierarchy: {
    name: string;
    type: "root" | "caller" | "dependency";
    file: string;
    children: CallNode[];
  };
  reasoning: string[];
}
```

06 Relational Database Schema (Prisma / SQLite)

The persistent database model defined in `backend/prisma/schema.prisma` incorporates 15 normalized tables ensuring comprehensive auditability:

Model	Primary Attributes	Relationships & Cascade Rules
`Repository`	`id, name, url, defaultBranch, ownerId`	1-to-many with AnalysisRun, Contributor, Commit, Documentation, AISession.
`AnalysisRun`	`id, branch, commitHash, status, healthScore, archScore`	Belongs to Repository (Cascade). 1-to-many with File, Dependency, TechDebt, RiskScore.
`File`	`id, path, name, extension, language, loc, complexity`	Belongs to AnalysisRun. 1-to-many with ClassDefinition, FunctionDefinition, Dependencies.
`Dependency`	`id, fromFileId, toFileId, importStatement, isCircular`	Relates source file to target file. Supports graph reconstruction and coupling queries.
`TechnicalDebt`	`id, severity, component, file, issue, category, recommendation`	Structured audit log tracking complexity hotspots, dead code, and security alerts.
`RiskScore`	`id, fileId, complexity, changeFrequency, riskScore, riskLevel`	Synthesizes Git commit frequency with cyclomatic complexity.
`Commit`	`id, hash, author, message, timestamp, linesAdded, linesDeleted`	Stores full historical repository commits and file churn changes.

07 Frontend Architecture & User Experience

The frontend uses Next.js 16.3 with React 19, styled with an editorial slate developer aesthetic. Every view is built for high information density without unnecessary decorative fluff:

1. Executive Overview (`OverviewView.tsx`)

Metric ribbon (LOC, Files, Commits), composite 68/100 Engineering Health Index with progress bars, Immediate Attention banner highlighting circular loops and security deprecations.

2. Architecture Canvas (`ArchitectureView.tsx`)

Interactive React Flow graph with custom service nodes, animated circular dependency edges, role filters (Services, Controllers, Repositories, Database), and node inspector drawer.

3. Dependencies Explorer (`DependenciesView.tsx`)

Comprehensive coupling matrix displaying fan-in (dependents), fan-out (depends-on), and circular loops to expose architectural single points of failure.

4. Codebase Explorer (`CodebaseView.tsx`)

VS-Code inspired 3-pane layout: Directory tree, full Monaco Editor with syntax highlighting and minimap, and AST symbol inspector with direct Blast Radius triggers.

5. Git Churn Dynamics (`GitHistoryView.tsx`)

Milestone timeline (2021–2024), most volatile files ranking (change frequency), and full commit logs with author signatures and hashes.

6. Technical Debt Catalog (`TechDebtView.tsx`)

Filterable table classifying debt into Complexity, Circular Dependencies, Dead Code, and Security vulnerabilities with actionable remediation plans.

7. Risk Matrix (`RiskAnalysisView.tsx`)

Interactive Recharts scatter plot charting Change Frequency (X-axis) against Cyclomatic Complexity (Y-axis). Accurately isolates the Critical Hotspot quadrant.

8. Contextual AI Assistant (`AIAssistantView.tsx`)

3-column layout: Active Index Context, conversation thread, and verified Citation Inspector citing file lines, snippets, and Git hashes.

08 Bundled Legacy Codebase: `legacy-payment-system`

To ensure realistic demonstration out of the box, the project bundles a realistic production-like legacy TypeScript codebase at `demo-repo/legacy-payment-system` illustrating canonical architectural anti-patterns:

Component	File Path	Anti-Pattern & Architectural Finding
`PaymentService`	`src/payment/PaymentService.ts`	CRITICAL Cyclomatic Complexity 18; nested gateway branches; circular import to `TransactionService`; direct Singleton coupling.
`TransactionService`	`src/payment/TransactionService.ts`	HIGH Reciprocal dependency: imports `PaymentService` to invoke `refundPayment`, completing circular loop.
`AuthController`	`src/auth/AuthController.ts`	HIGH Zero test coverage; imports deprecated legacy MD5 encryption routine for password verification.
`DatabaseConnector`	`src/database/DatabaseConnector.ts`	MEDIUM Global Singleton architectural hub with 6 direct dependents; high blast radius on connection failure.
`LegacyCryptoUtils`	`src/utils/LegacyCryptoUtils.ts`	MEDIUM Dead code candidates (`unused_rot13_obfuscate` , `legacy_xor_cipher`); insecure MD5 implementation.

09 Execution & Developer Operations Guide

Backend Service (Port 5000)

```
cd backend
npm install
npm run dev

# API available at: http://localhost:5000/api
# Health check:     http://localhost:5000/api/health
```

Frontend Next.js App (Port 3000)

```
cd frontend
npm install
npm run dev

# Web Application: http://localhost:3000
```

Direct Python Analysis Execution

```
# Run standalone engine on the bundled demo repository:
python analysis-engine/analyzer.py demo-repo/legacy-payment-system

# Or run via root package.json shortcut:
npm run analyze:demo
```